

REPORT

제목:



■ 과목명

딥러닝 응용

■ 교수명

김선만 교수님

■ 학과명

소프트웨어융합학부

■ 학 번

202155011

■ 이 름

송영욱

■ 제출일

2025-10-28

숫자 분류기 구현하기

-코드 제시

```
import torch
import torch.nn as nn
import torch.nn.functional as F
#%%
class CNN(nn.Module):
    def __init__(self):
        super().__init__()
        # First Convolutional Layer (Depthwise Pointwise)
        self.dw_conv1 = nn.Conv2d(in_channels=1, out_channels=1, kernel_size=2, stride=1,
padding=1, groups=1)
        self.pw_conv1 = nn.Conv2d(in_channels=1, out_channels=32, kernel_size=1)
        self.bn1 = nn.BatchNorm2d(32)
        self.pool1 = nn.MaxPool2d(kernel_size=2, stride=2) # Using MaxPool

        # Second Convolutional Layer (Depthwise Pointwise)
        self.dw_conv2 = nn.Conv2d(in_channels=32, out_channels=32, kernel_size=4, stride=1,
padding=1, groups=32)
        self.pw_conv2 = nn.Conv2d(in_channels=32, out_channels=64, kernel_size=1)
        self.bn2 = nn.BatchNorm2d(64)
        self.pool2 = nn.MaxPool2d(kernel_size=2, stride=2) # Using MaxPool

        # third Convolutional Layer (Depthwise Poinrwise)
        self.dw_conv3 = nn.Conv2d(in_channels=64, out_channels=64, kernel_size=4, stride=1,
padding=1, groups=64)
        self.pw_conv3 = nn.Conv2d(in_channels=64, out_channels=128, kernel_size=1)
        self.bn3 = nn.BatchNorm2d(128)
        self.pool3 = nn.MaxPool2d(kernel_size=2, stride=2) # Using MaxPool

        # Classifier head with Global Average Pooling
        self.gap = nn.AdaptiveAvgPool2d((1, 1))
        self.dropout = nn.Dropout(0.25)
        self.fc = nn.Linear(128, 10)

    def forward(self, x):
        x = x.view(-1, 1, 28, 28)

        # Conv layers
        x = self.dw_conv1(x)
        x = self.pw_conv1(x)
        x = self.bn1(x)
        x = F.relu(x)
        x = self.pool1(x)
```

```

        x = self.dw_conv2(x)
        x = self.pw_conv2(x)
        x = self.bn2(x)
        x = F.relu(x)
        x = self.pool2(x)

        x = self.dw_conv3(x)
        x = self.pw_conv3(x)
        x = self.bn3(x)
        x = F.relu(x)
        x = self.pool3(x)

        # Classifier
        x = self.gap(x)
        x = torch.flatten(x, 1)
        x = self.dropout(x)
        x = self.fc(x)
        return x
#%%
if __name__ == "__main__":
    # 데이터 테스트
    batch_size = 64
    x = torch.randn(batch_size, 1, 28, 28)

    model = CNN()
    output = model(x)

    # 모델 파라미터 수 계산
    num_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
    print(f"Number of trainable parameters: {num_params:,}")

    # 모델 파라미터 및 연산량(MACs) 계산 pip install thop
    try:
        from thop import profile
        macs, params = profile(model, inputs=(torch.randn(1, 1, 28, 28), ))
        print(f'MACs: {macs / 1e9:.5f} G, Params: {params / 1e6:.5f} M')
    except ImportError:
        print("thop is not installed. Please install it using 'pip install thop'")

    print(f"Input shape: {x.shape}")
    print(f"Output shape: {output.shape}")

```

-모델 구조 설계안 제시

본 프로젝트에서는 MNIST 손글씨 숫자 이미지 분류를 위한 경량 Convolutional Neural Network(CNN)를 설계하였습니다. 본 모델은 일반적인 합성곱 신경망과 달리 Depthwise Separable Convolution 구조를 채택하였습니다. 그 이유는 제가 Depthwise Convolution과 Pointwise Convolution을 잘 모르는 것 같아서 공부해 보았습니다.

모델은 총 3개의 Covolution Layer로 구성했습니다. 각 블록은 Depthwise Convolution과 Pointwise Convolution으로 이루어져 있습니다. Depthwise Convolution은 입력 채널별로 필터를 적용하고, 이어지는 Pointwise Convolution(1×1 convolution)을 통해 채널 수를 확장합니다. 각 Convolution layer에는 Batch Normalization과 ReLU 활성화 함수가 적용되며, MaxPooling을 사용하여 공간 크기를 점진적으로 줄입니다.

세 개의 Convolution Layer를 통과한 후, 해당 특성 맵에 대해 Global Average Pooling을 적용하여 채널별 특징을 하나의 값으로 요약합니다. 이후 Dropout을 적용하여 학습 시 과도한 패턴 기억을 억제하고 일반화 성능을 높였다. 최종적으로 Linear Layer를 통해 10개의 클래스(MNIST의 숫자 0~9)에 대한 Logit 값을 출력합니다.

학습 과정에서는 CrossEntropyLoss를 손실 함수로 사용하였으며, 최적화는 Adam Optimizer를 사용하였다. 초기 학습률(Learning Rate)은 0.001로 설정하였고, Batch Size는 64로 구성하였다. Epoch 수는 10으로 설정하여 학습을 진행하였습니다.

-모델 구조 설계

Task2이며 연산량은 0.000886 이고 파라미터수는 0.01387로 제한하신 수를 준수하였습니다.

```
PS C:\Users\syu10\Desktop\TASK> & C:/Users/syu10/anaconda3/python.exe c:/Users/syu10/Desktop/TASK/Task2.py
Number of trainable parameters: 13,871
[INFO] Register count_convNd() for <class 'torch.nn.modules.conv.Conv2d'>.
[INFO] Register count_normalization() for <class 'torch.nn.modules.batchnorm.BatchNorm2d'>.
[INFO] Register zero_ops() for <class 'torch.nn.modules.pooling.MaxPool2d'>.
[INFO] Register count_adap_avgpool() for <class 'torch.nn.modules.pooling.AdaptiveAvgPool2d'>.
[INFO] Register zero_ops() for <class 'torch.nn.modules.dropout.Dropout'>.
[INFO] Register count_linear() for <class 'torch.nn.modules.linear.Linear'>.
MACs: 0.00086 G, Params: 0.01387 M
Input shape: torch.Size([64, 1, 28, 28])
Output shape: torch.Size([64, 10])
PS C:\Users\syu10\Desktop\TASK>
```

-최종 결과

